SDU

Real Time Operating Systems
FreeRTOS
Pre-emptive Scheduling

Timeline for today

- Lectures
  - FreeRTOS
  - Pre-emptive scheduling
- Intro to Assignment07, selection of groups
- Lab8
  - How to get started with FreeRTOS in Code Composer Studio
  - FreeRTOS template
  - Lab08 task
  - Lab08 solution provided

Course outline

- **The plan is indicative, subject to change!**

Date	Lecture	Subject(s)	Lab	Assignment due
Feb 2	1	Introduction; ARM Cortex-M4	Lab1: Setting up the development env.	
Feb 9	2	EMP coding standard; Bit manipulation	Lab2: Bit manipulation	
Feb 16	3	State machines; The compiler chain; EMP-Board	Lab3: State machines	PF1 – Assignment 1
Feb 23	4	The task model; The pre-processor	Lab4: A clock radio task model	PF1 – Assignment 2
Mar 2	5	Queues and semaphores; Debugging	Lab5: Debug with a serial connection	PF1 – Assignment 3
Mar 9	6	Run to complete scheduler	Lab6: RTCS, Run to compile scheduler	PF1 – Assignment 4
Mar 16	7	More debugging; C: printf()	Lab7: RTCS, Debugger	PF1 – Assignment 5
March 23	8	FreeRTOS	Lab8: FreeRTOS	PF1 – Assignment 6
March 30 April 6		Easter holiday		
Apr 13	9	More queues; Assembler in C	Lab9: FreeRTOS (continued)	PF1 – Assignment 7
Apr 20	10	Re-entrance		PF1 – Assignment 8
Apr 27	11	Work on the final assignment, consultations		
May 4	12	Poster session		PF2- Final Assignment

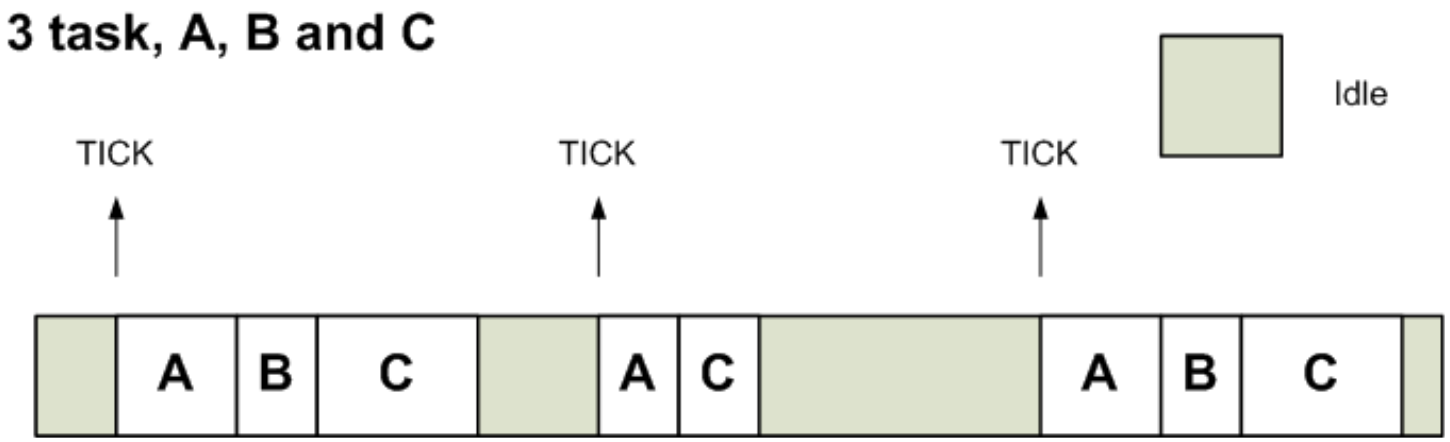
Poster presentation

- For final assignment you submit your code and a poster in PDF
- One poster per group
- Should be A0, but A1 is also acceptable
- Use SDU plotters and SDU-bucks for cost-efficiency
- Put up your posters on May 19th in ??? on the walls/windows
- Poster content ideas:
  - Similar to weekly assignment presentation slides but in one
  - Task diagram, state machines
  - Photos of your system
  - Renderings of how the user interface works
  - QR code to a video
  - Pieces of interesting code
- Bring your TIVA and EMP boards for a live demo!

SDU

Recap - Run to Complete Scheduling

- Each task runs until it is finished its job
- Scheduling is based on the tick
- The tick is the basic unit of time for task scheduling
- An example with 3 tasks
- Cooperative scheduling – what happens if a task doesn't exit?



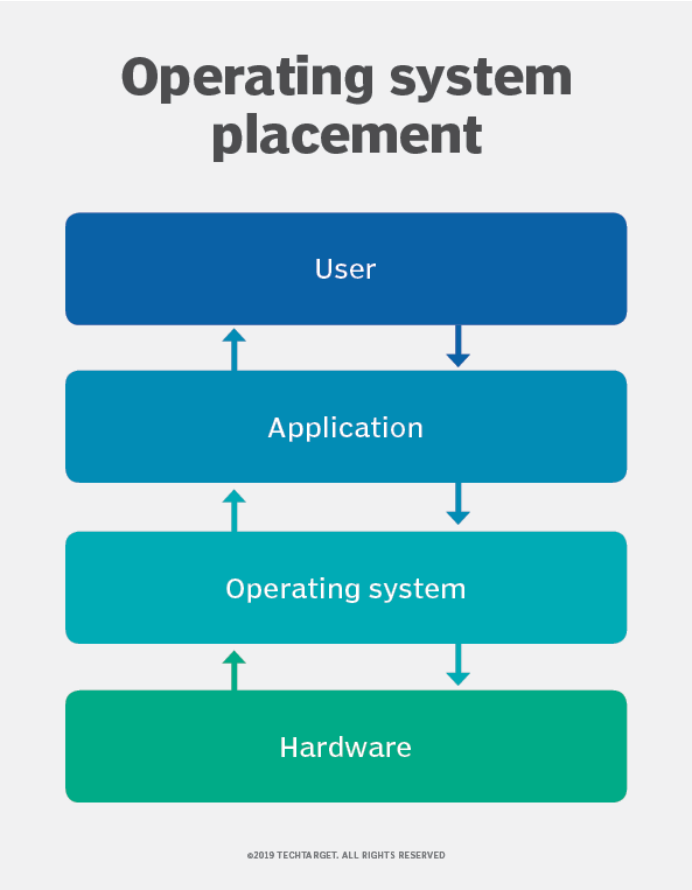
Free RT OS

- OS
- RTOS
- FreeRTOS

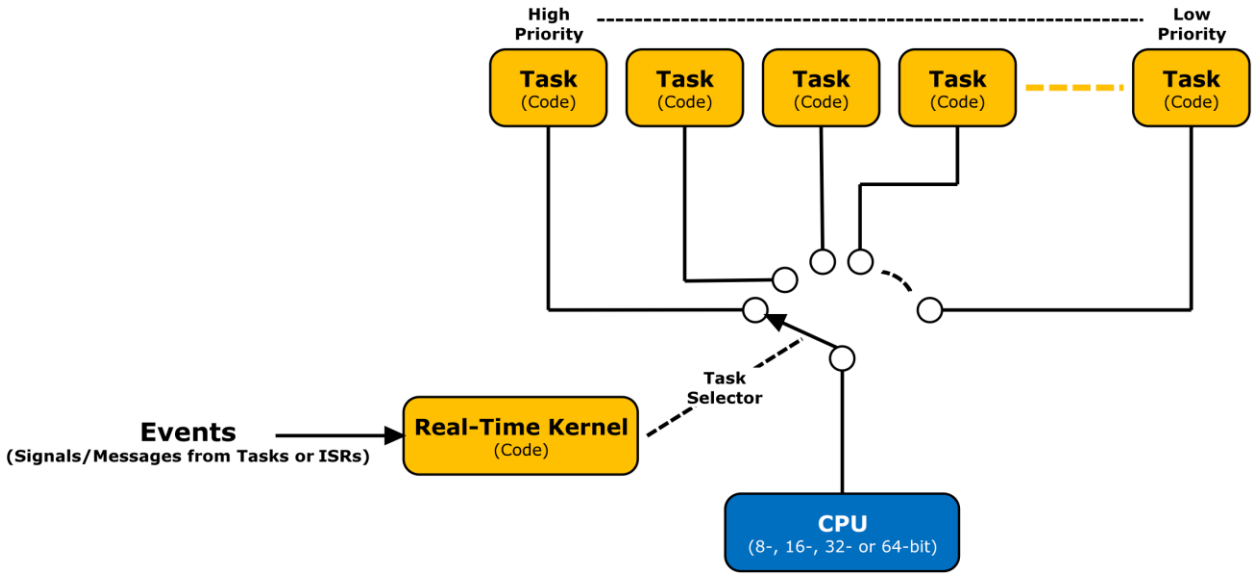
OS – operating system

- Computer program that supports the computer's basic functions
- Provides services to other programs (applications)

- Multi-tasking – scheduler's reponsibility
- On single core processors illusion of simultaneous execution by rapid switching
- Most OS are not real-time: Windows, most Linuxes

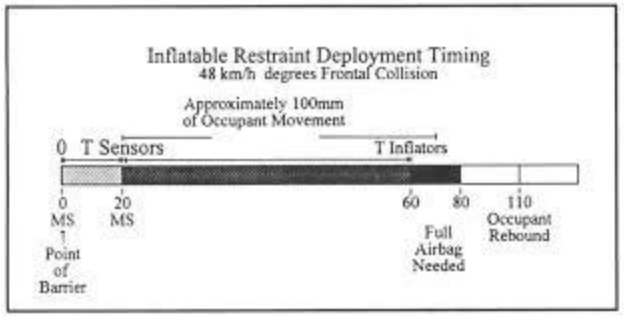


RTOS



- Real-time operating system
- Don't confuse with RTCS – run to complete scheduling
- Responds to events within a strictly defined time (deadline)
- Scheduler is deterministic
- Of interest to embedded systems especially
- Priority assigned to each thread
- Threads = tasks
- process data as it comes in, typically without buffer delays

Applications of RTOS



- Where meeting deadlines is critical for task completion
- The results only make sense if delivered within time constraints
- E.g.
  - In car systems – airbag deployment
  - Air defense systems – military systems in general
  - Traffic control systems
  - Process control in industry – temperature, pressure, flow
    - oil industry, food industry
  - Closed loop control systems in general – air conditioners, thermostats
  - Digital audio decoding – MP3
  - Machine vision guided robotics

FreeRTOS

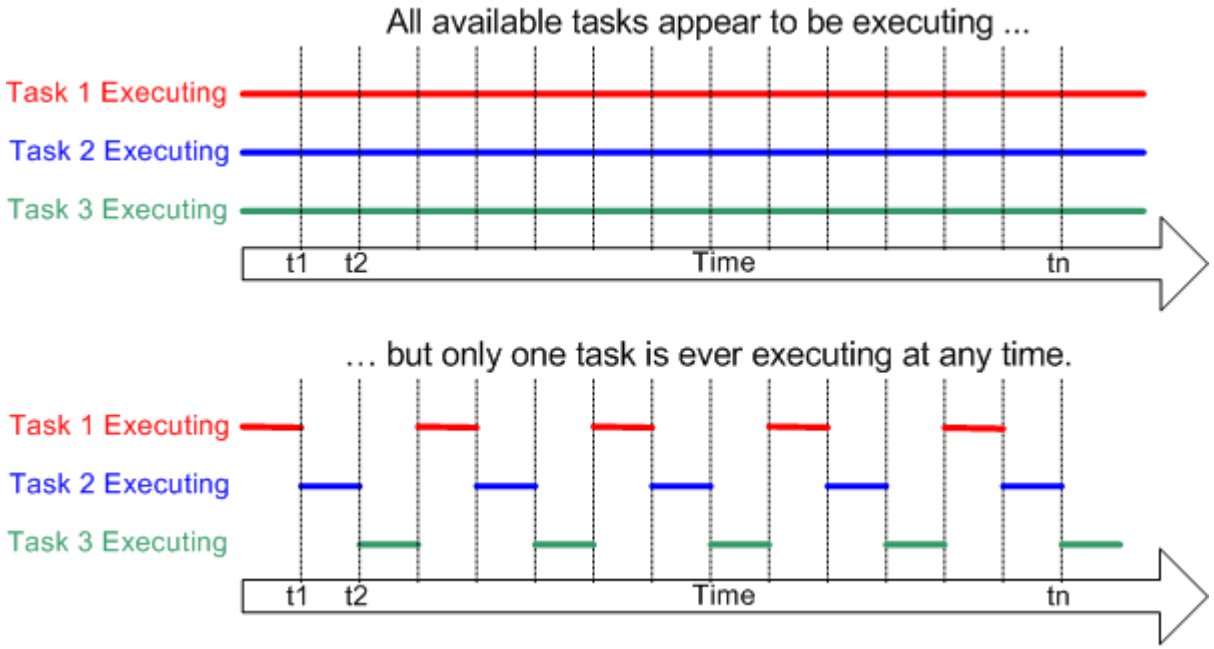
- RTOS small enough for microcontrollers
- Embedded controllers don’t warrant full RTOS because of their hardware limitation
- FreeRTOS provides only:
  - Real-time scheduling functionality
  - Inter-task communication
  - Timing and synchronization primitives
- FreeRTOS – more a real-time kernel than full OS

FreeRTOS in their creators' words

- Market leading
- Professionally developed
- Quality controlled
- Robust
- Supported
- Free to use in commercial products without requirement to expose proprietary source code!
- Now owned by Amazon Web Services

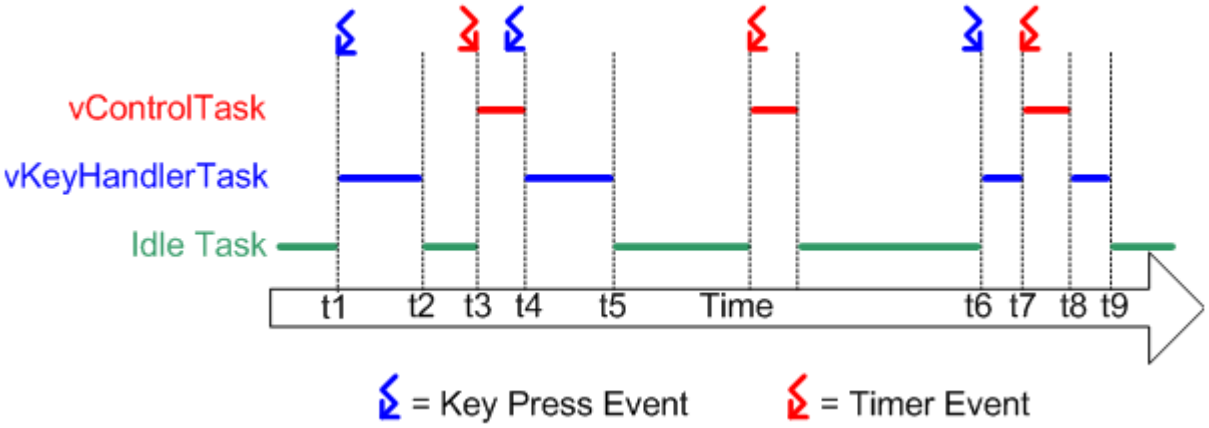
Multi-tasking

- Kernel – core of OS
- If it can execute tasks seemingly simultaneously – multi-tasking
- Allows complex applications to be subdivided into small tasks



Scheduler

- Part of the kernel deciding which task will be executed when
- The scheduler can suspend and resume tasks
- Tasks can suspend themselves too



FreeRTOS choice od scheduling policy

- Pre-emptive
  - Always runs the highest available task. Tasks of identical priority share CPU time (fully pre-emptive with round robin time slicing).
- Co-operative
  - Context switches only occur if a task blocks or explicitly calls taskYIELD()

Context switching

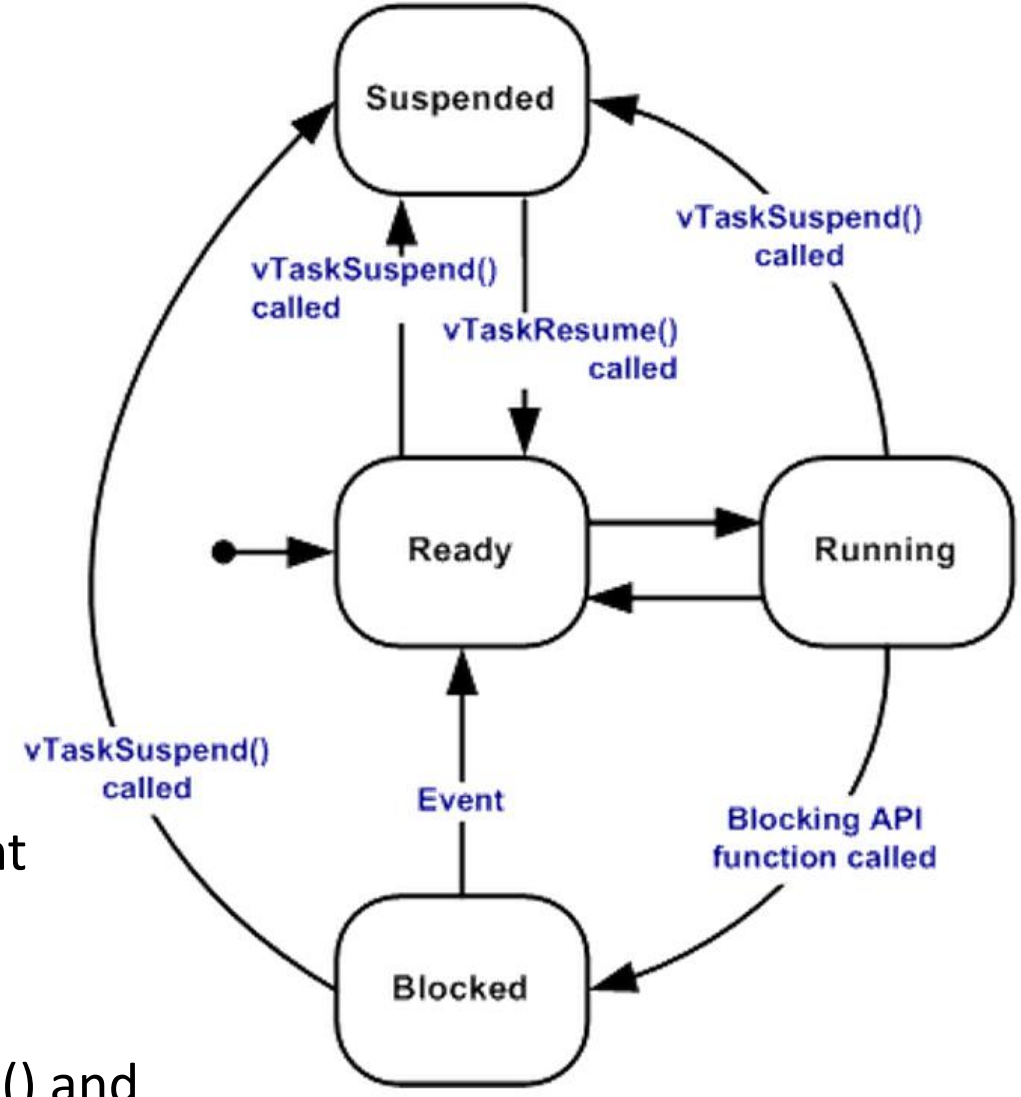
- Tasks use different resources of the microcontroller – registers, stack, etc. – context
- When the kernel switches tasks, it saves its context
- When a task is resumed, the context is restored

FreeRTOS additional features

- Message queues
- Binary and counting semaphores
- Mutexes

FreeRTOS task states

- Running
  - Actually executing – using the processor
- Ready
  - Able to execute (not blocked nor suspended)
  - Not executing because another task with equal or higher priority is already running
- Blocked
  - Waiting for a temporal or external event
  - Can block waiting for a queue or semaphore event
- Suspended
  - Not available for scheduling
  - Tasks only enter or exit suspend by vTaskSuspend() and xTaskResume()



Valid task state transitions

Task API

- **xTaskCreate()**
- **xTaskDelete()**
- **vTaskDelay()**
- **TaskYIELD()**

```
// Task to be created.
void vTaskCode( void * pvParameters )
{
        initialize();
        while( 1 )
        {
                // Task code goes here.
        }
}
```

For deatils on FreeRTOS funcions see API Reference here

Creating and deleting tasks

```
// Function that creates a task.
void some_function( void )
{
	static unsigned char ucParameterToPass;
	xTaskHandle xHandle;

	// Create the task, storing the handle.
	xTaskCreate( vTaskCode, "NAME", STACK_SIZE,
&ucParameterToPass, tskIDLE_PRIORITY, &xHandle );

	// Use the handle to delete the task.
	vTaskDelete( xHandle );
}
```

Delaying a task

```
void task_code( void * pvParameters )
{
	/* Block for 500ms. */
	const portTickType xDelay = 500 / portTICK_RATE_MS;
	while( 1 )
	{
		/* Simply toggle the LED every 500ms,blocking between
each toggle. */
		toggle_LED();
		vTaskDelay( xDelay );
	}
}
```

Semaphores

- Semaphores are normally used for signaling or synchronization between tasks and between tasks an interrupt service routines.
- A semaphore can be a signaling semaphore or a mutex.
- A semaphore can be binary or counting.

Semaphores API

- Binary semaphores
  - xSemaphore = vSemaphoreCreateBinary( );
- Counting semaphores
  - xSemaphore = xSemaphoreCreateCounting(uxMaxCount, uxInitialCount);
- Semaphore operations
  - xSemaphoreGive(xSemaphore);
  - xSemaphoreTake(xSemaphore, *xBlockTime*);

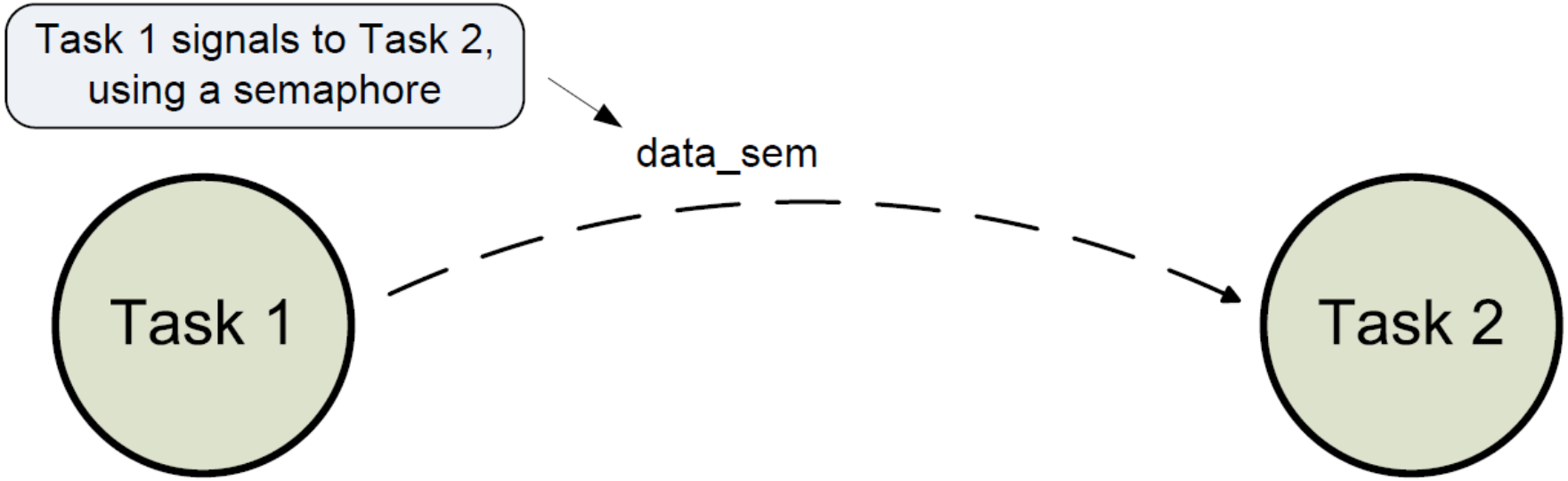
Creating a semaphore

- Before a semaphore is used it must be explicitly created

```
SemaphoreHandle_t data_sem;

void vATask( void * pvParameters )
{
        /* Semaphore cannot be used before a call to
                SemaphoreCreateBinary (). */
        data_sem = xSemaphoreCreateBinary( void );
        if( data_sem != NULL )
        {
                /* The semaphore was created successfully.
                The semaphore can now be used. */
        }
}
```

Semaphore example



Task 1
/* Give data_sem semaphore
to unblock task2. */
xSemaphoreGive(data_sem);

Task 2
/* wait for data_sem semaphore.
The task blocks indefinitely until data_sem has been given */
xSemaphoreTake(data_sem, portMAX_DELAY);

Waiting for a semaphore

```
if( xSemaphoreTake( xSemaphore, MILLI_SEC(5000)))
{
        // Use Resource
        // ..
        // ..
        xSemaphoreGive( xSemaphore );
}
```

Resource Management

- There is a potential risk for a conflict to arise in a multitasking system if one task starts to access a resource but does not complete its access before being transitioned out of the Running state. If the task left the resource in an inconsistent state then access to the same resource by any other task or interrupt could result in data corruption.

Mutual exclusion

- Access to a resource that is shared either between tasks or between tasks and interrupts needs to be managed using ‘mutual exclusion’.
- FreeRTOS provides several features that can be used to implement mutual exclusion
  - Disabling of interrupts.
  - Disabling the scheduler
  - Using Mutexes

Disabling of interrupts

- Prevents interrupts and the preemptive scheduler to pick another task.
- Increase interrupt latency
- Reduce the responsiveness of the system
- FreeRTOS API
  - taskENTER_CRITICAL()
  - taskEXIT_CRITICAL()

Disabling the scheduler

- A critical section implemented by suspending the scheduler only protects a region of code from access by other tasks because interrupts remain enabled
- FreeRTOS API
  - vTaskSuspendAll()
  - vTaskResumeAll()

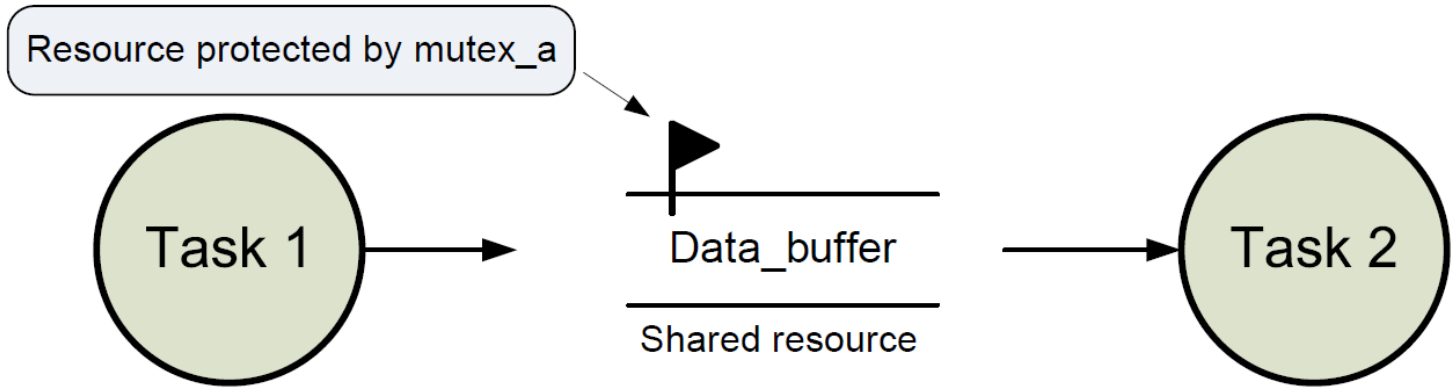
Mutex

- A Mutex is a special type of binary semaphore that is used to control access to a resource that is shared between two or more tasks. The word MUTEX originates from “MUTual EXclusion".
- A task which takes a Mutex, owns the Mutexand the task MUST always return it again.
- Mutex vs. binary semaphore – mutexes can be released only by the task which took them, while binary semaphores can be released by any task – the purpose of the the two are different – mutexes for protecting resource – binary semaphores for serialization, signaling

Mutex API

- Mutexes
  - xSemaphoreCreateMutex()
  - xSemaphoreTake()
  - xSemaphoreGive()

Mutex example




```
if ( xSemaphoreTake( mutex_a, portMAX_DELAY ))
{
        /* The following line will only execute once
        the mutex has been successfully obtained. */
        /* access data_buffer */
        /* The mutex MUST be given back! */
        xSemaphoreGive( mutex_a);
}
```

Queues

- A queue can hold a finite number of fixed size data items. The maximum number of items a queue can hold is called its ‘length’. Both the length and the size of each data item are set when the queue is created.
- Writing data to a queue causes a byte for byte copy of the data to be stored in the queue itself. Reading data from a queue causes the copy of the data to be removed from the queue.

Queue API

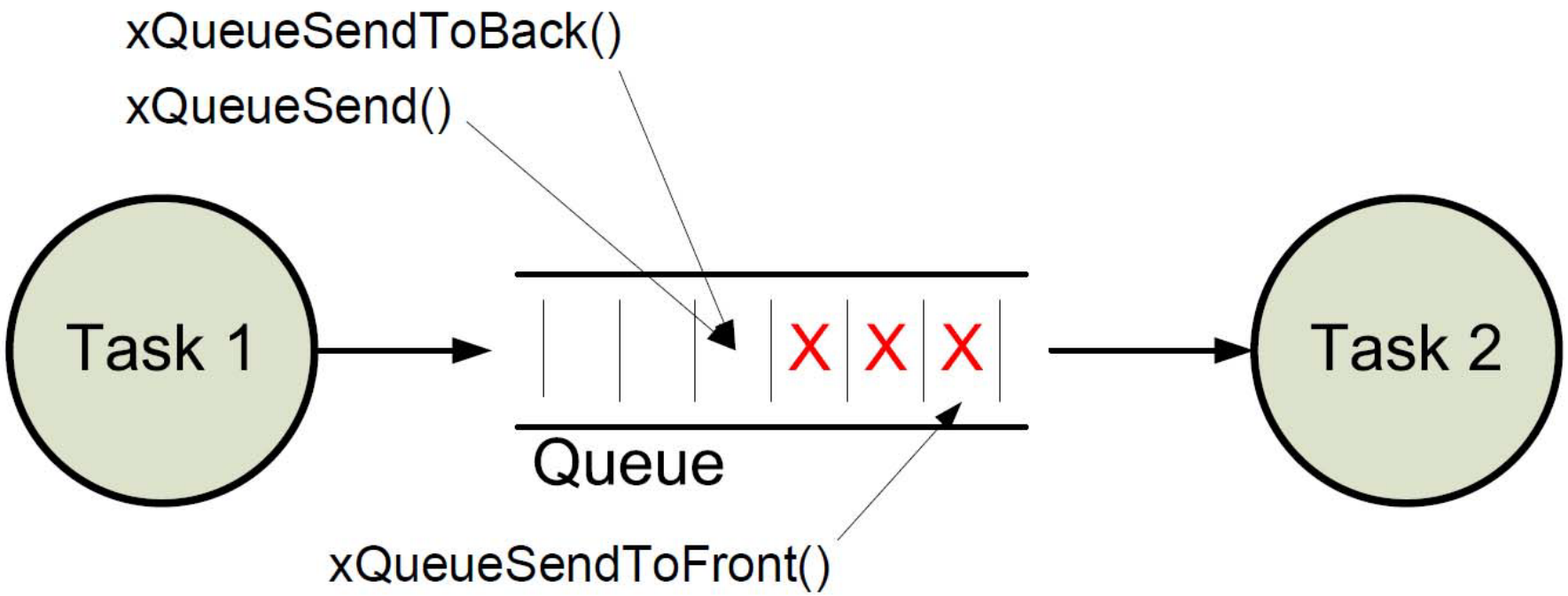
- xQueueHandle = xQueueCreate();
- xQueueSend();
- xQueueSendToBack();
- xQueueSendToFront();
- uxQueueMessagesWaiting();
- xQueueReceive();
- xQueuePeek();

Creating a queue

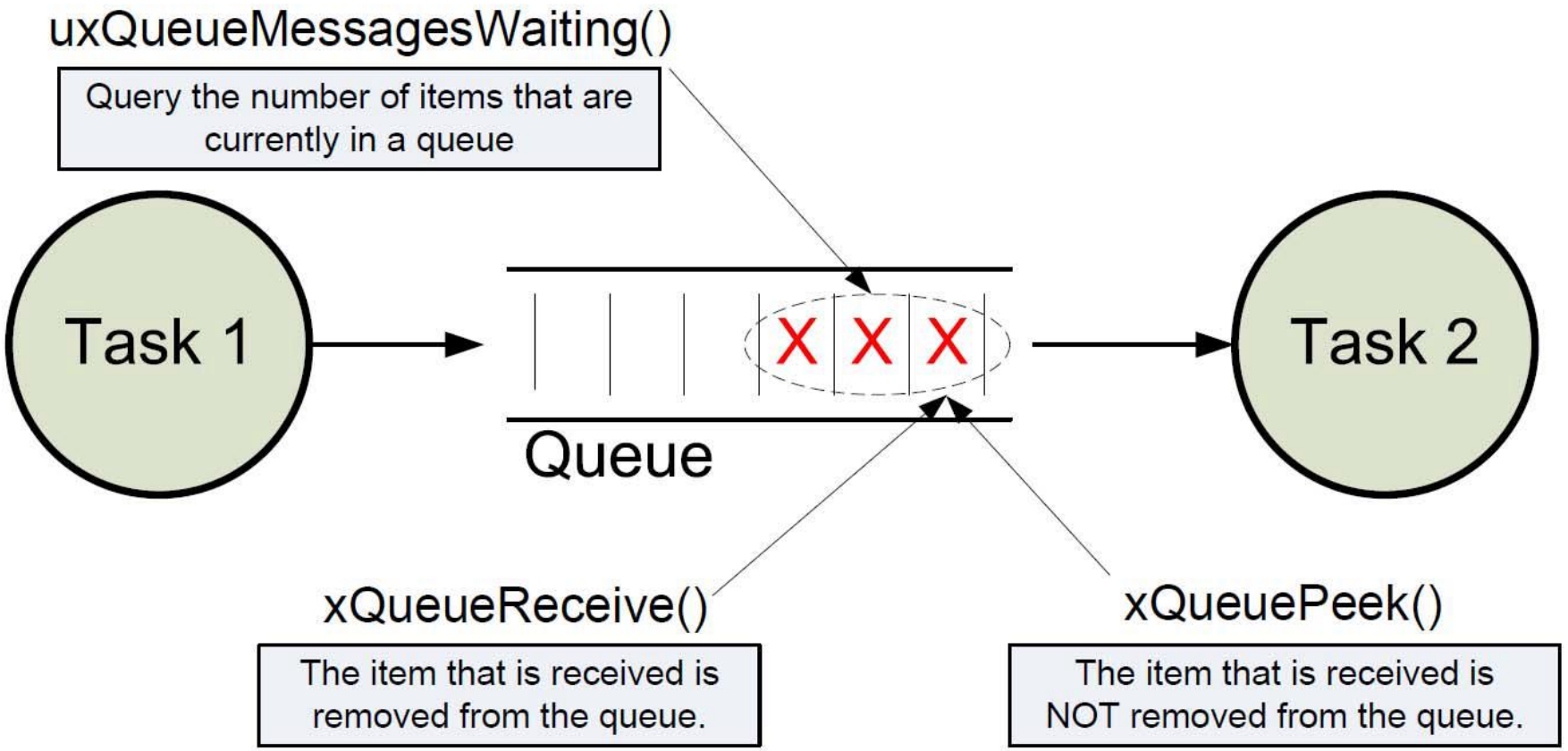
- A queue must be explicitly created before it can be used.
- xQueueCreate() is used to create a queue and returns an xQueueHandle to reference the queue it creates.
- xQueueCreate() will return NULL if there is insufficient memory for the queue to be created, otherwise a handler to the queue will be returned.

```
xQueueCreate(uxQueueLength, uxItemSize);
```

Put data in a queue



Receiving data from a queue



Interrupt routines

- Interrupt routines should in general not take a Semaphore or a Mutex.
- Only API functions and macros that end in ‘FromISR’ or ‘FROM_ISR’ should be used within an interrupt service routine.
- E.g.
  - xSemaphoreGiveFromISR

Pre-emptive Scheduling

Preemption

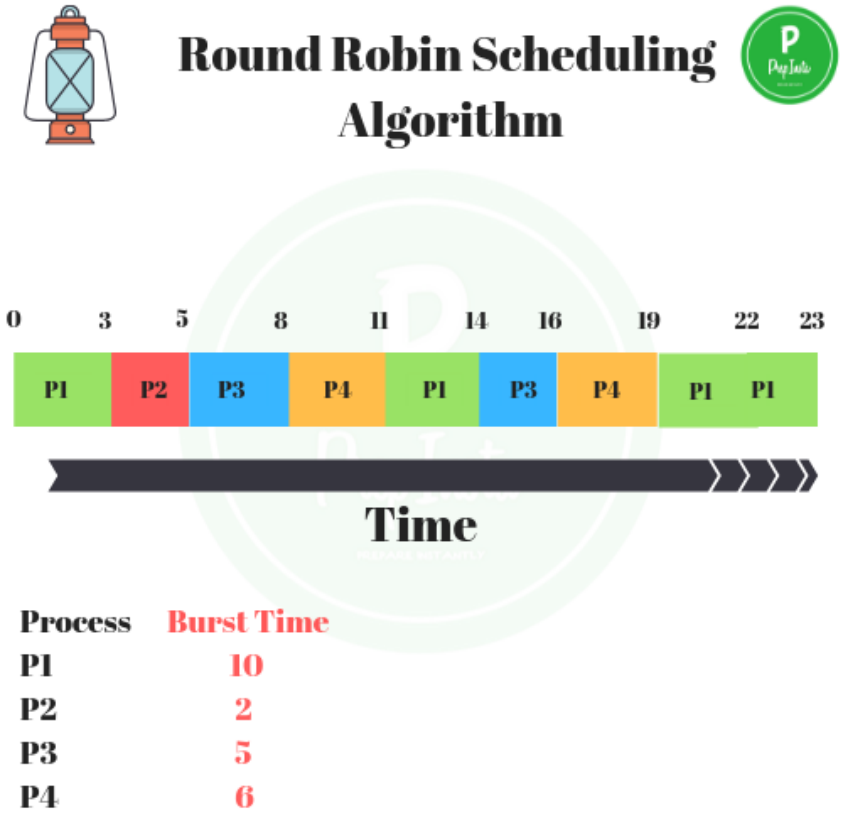
- General definition: a prior seizure or appropriation – taking possession before others
- Synonyms: protective, defensive, precautionary
- In computing:
  - interrupting a task without its cooperation,
  - but with the intention to resume it later

FreeRTOS scheduling policies

- Co-operative (non-preemptive)
  - The OS never takes control from a task
  - Context switches only occur if a task blocks or explicitly calls taskYIELD()
- Pre-emptive (non-cooperative)
  - Always runs the task with highest priority
  - Tasks of identical priority share CPU time
    - fully pre-emptive with round robin time slicing

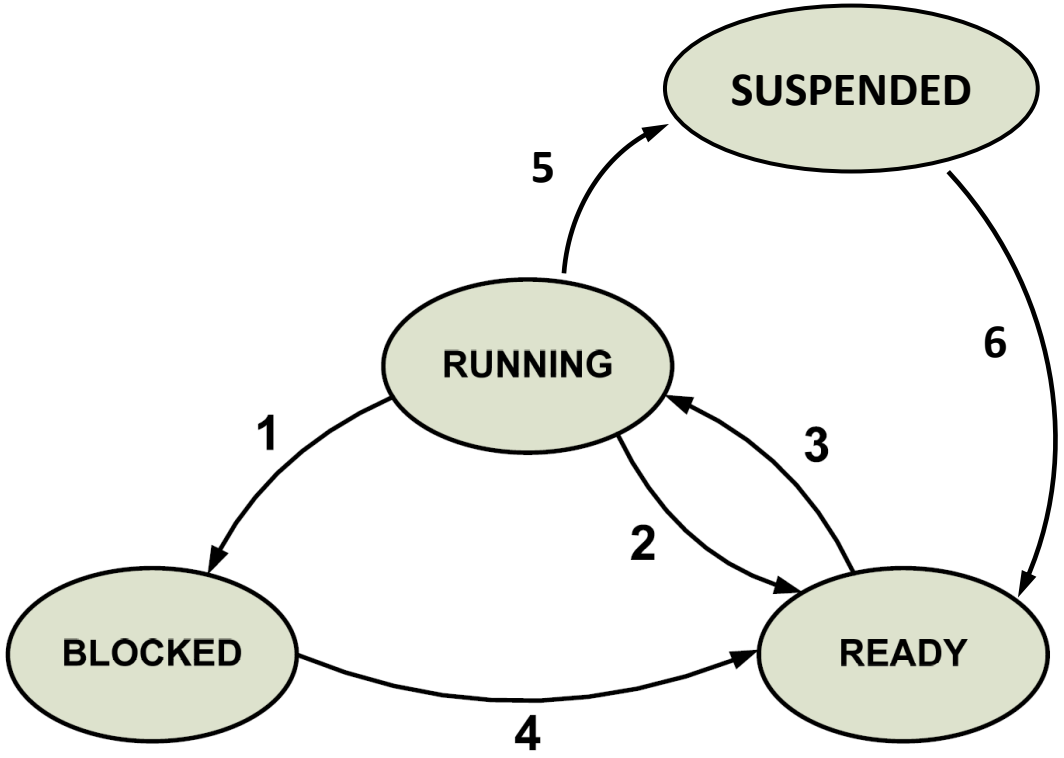
Round-robin

- In sports: every team plays once against every other team
- Each task (with same priority) assigned equal time slot
- Simple
- Makes sure that all tasks are served
- The task is resumed the next time it is scheduled

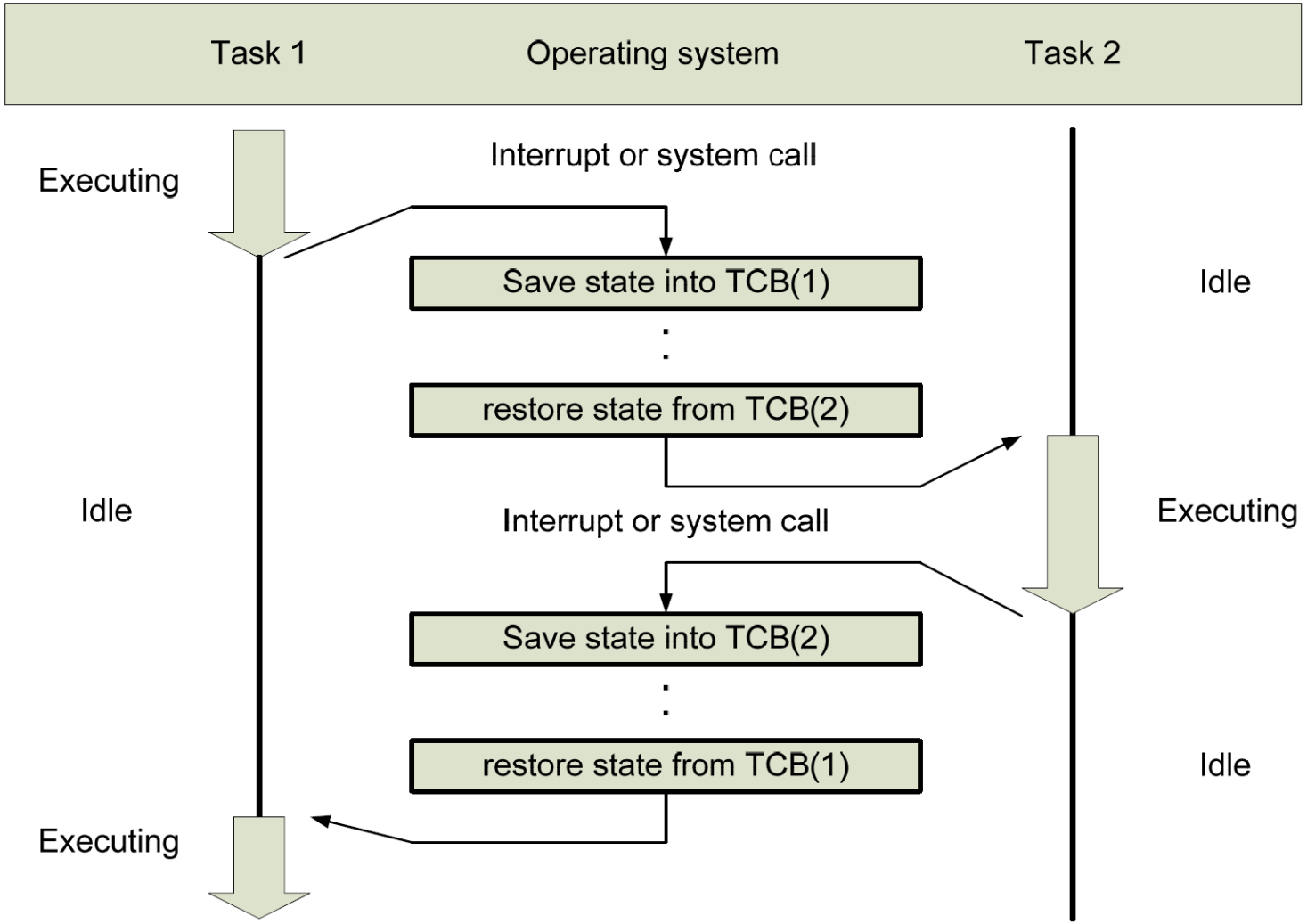


Task states

- 1: Task blocks for input or event.
- 2: Scheduler picks another task.
- 3: Scheduler picks this task
- 4: Input or event becomes available
- 5: vTaskSuspend() called
- 6: vTaskResume() called



Switching from task to task



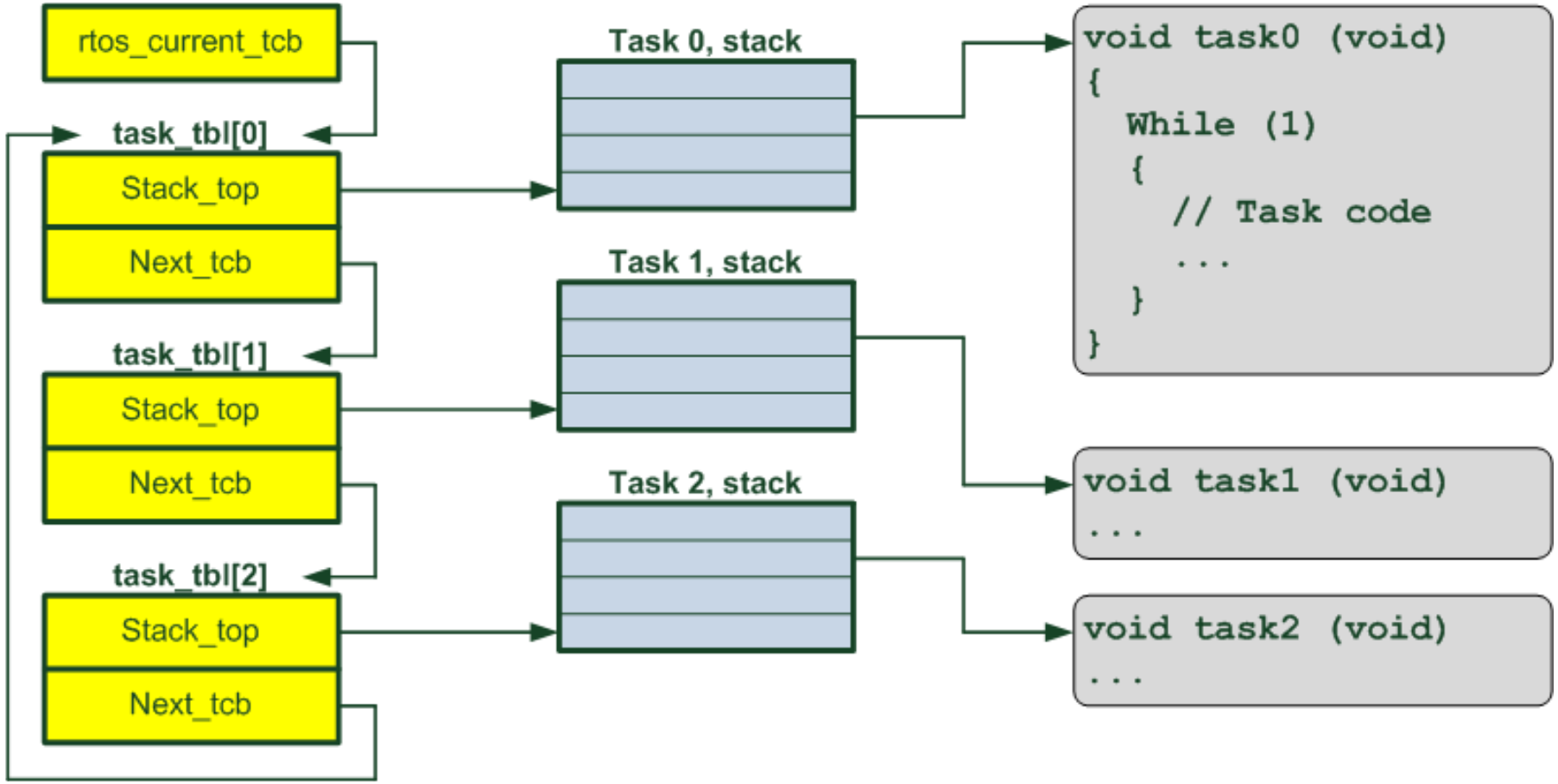
Task control block (TCB)

- For each task, the kernel controls:
  - ID
  - Program counter
  - Registers
  - Stack pointer
  - State
  - Priority
  - Timer, semaphore, queue
  - Task name
  - Task parameters
  - Etc.

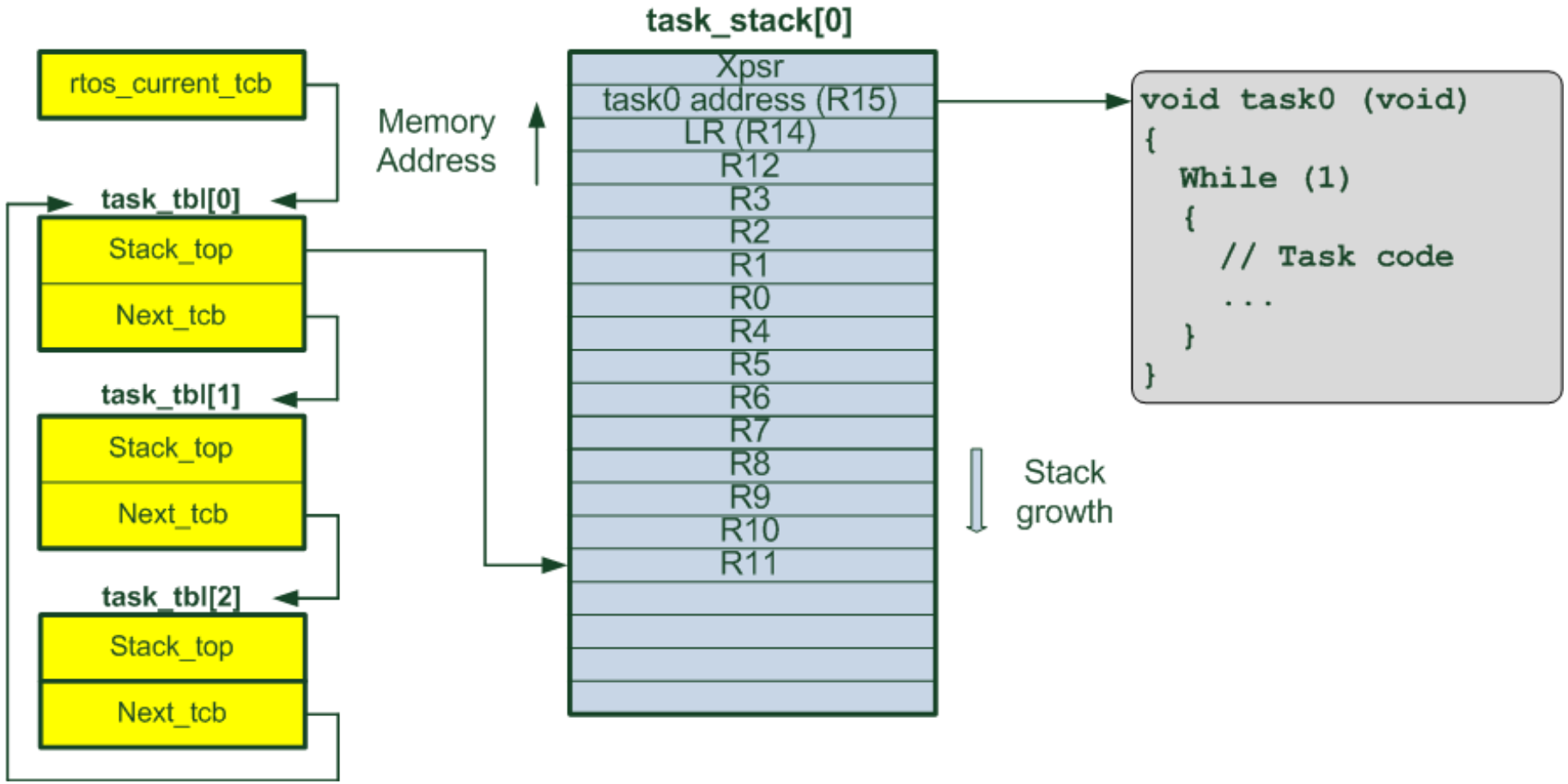
Context switching

- Tasks use different resources of the microcontroller – registers, stack, etc. – context
- When the kernel switches tasks, it saves its context
- When a task is resumed, the context is restored

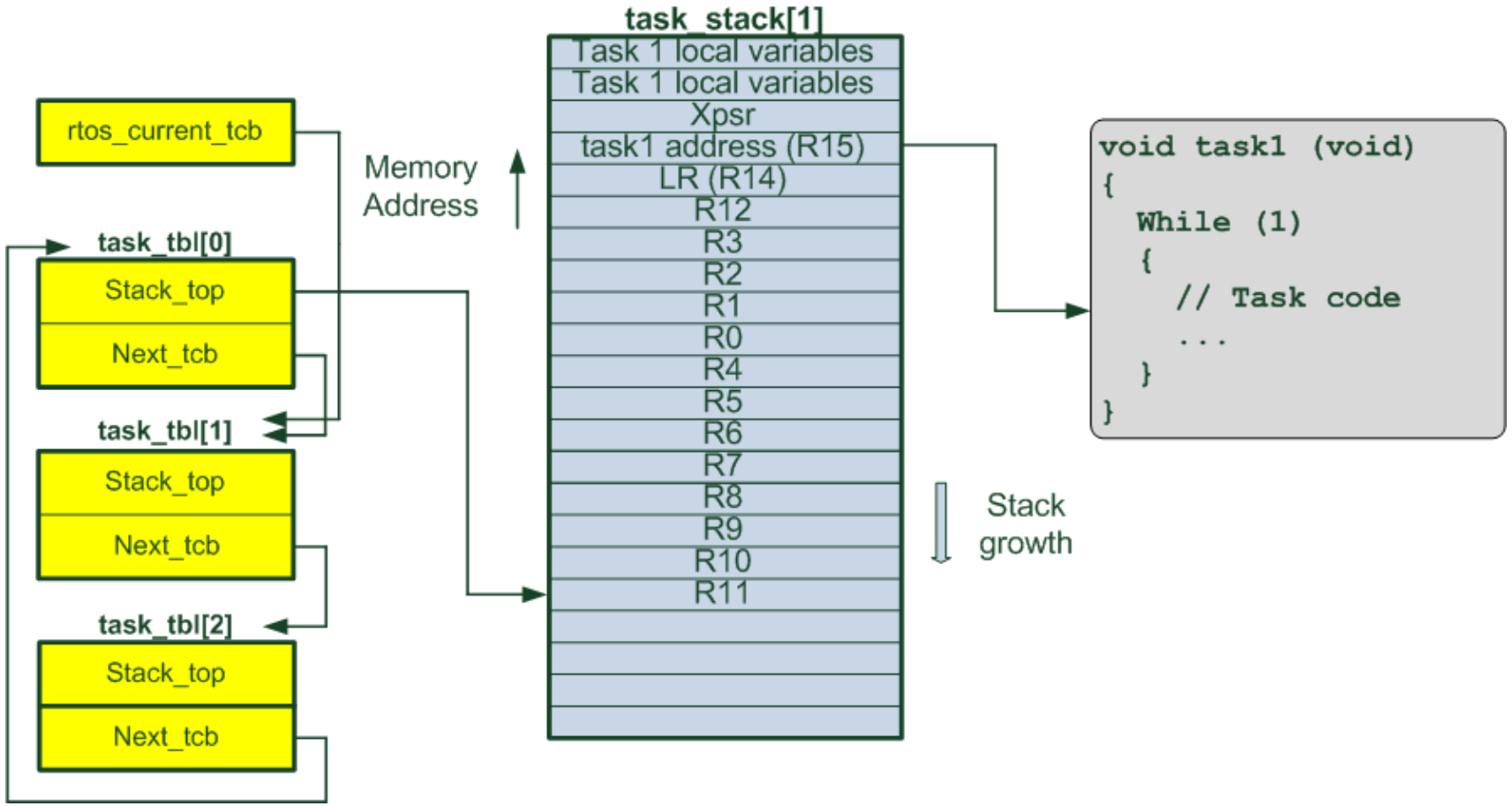
System after initial setup



After task 0 stack initialization



The scheduler picks task 1



Simple example from FreeRTOS.org

```
void main_blinky( void )
{

            /* The full demo configures the clocks for maximum frequency, wheras this blinky demo uses a slower clock as it also uses low power features. */
            prvConfigureClocks();

            /* Create the queue. */
            xQueue = xQueueCreate( mainQUEUE_LENGTH, sizeof( uint32_t ) );

            if( xQueue != NULL )
            {
                        /* Start the two tasks as described in the comments at the top of this file. */
                        xTaskCreate( prvQueueReceiveTask,/* The function that implements the task. */
                        "Rx", /* The text name assigned to the task - for debug only as it is not used by the kernel. */
                        configMINIMAL_STACK_SIZE, /* The size of the stack to allocate to the task. */
                        ( void * ) mainQUEUE_RECEIVE_PARAMETER, /* The parameter passed to the task - just to check the functionality. */
                        mainQUEUE_RECEIVE_TASK_PRIORITY, /* The priority assigned to the task. */
                        NULL );/* The task handle is not required, so NULL is passed. */

                        xTaskCreate(prvQueueSendTask,"TX",configMINIMAL_STACK_SIZE,(void *) mainQUEUE_SEND_PARAMETER, mainQUEUE_SEND_TASK_PRIORITY, NULL );

            /* Start the tasks and timer running. */
            vTaskStartScheduler();
            }
/* If all is well, the scheduler will now be running, and the following line will never be reached.  If the following line does execute, then
there was insufficient FreeRTOS heap memory available for the idle and/or timer tasks to be created.  See the memory management section on the
FreeRTOS web site for more details. */
            for( ;; );
}
```